

Instrukcja do napisania programu eksplorującego labirynt z wykorzystaniem OpenMP

1. Cel programu

Napisać program w C++, który realizuje równoległe eksplorowanie labiryntu. Labirynt jest reprezentowany jako tablica dwuwymiarowa (`tab`), w której:

- `-1` oznacza ścianę,
- `0` oznacza korytarz, który można odwiedzić.

Każde zadanie eksplorujące labirynt musi chronić operacje odczytu i zapisu za pomocą odpowiedniej blokady z tablicy (`lck`).

2. Przygotowanie projektu

1. Wymagania:

- Kompilator obsługujący OpenMP (np. `g++` lub `clang++`).
- Znajomość podstaw C++ i OpenMP.

2. Struktura programu:

- Tablica (`tab`) przechowuje mapę labiryntu.
 - Tablica blokad (`lck`) zabezpiecza dostęp do każdej komórki.
 - Funkcja `lab()` realizuje eksplorację labiryntu za pomocą zadań OpenMP.
 - Funkcja `view_tab()` drukuje stan labiryntu po zakończeniu eksploracji.
-

3. Kroki implementacji

Krok 1: Zainicjalizuj tablicę labiryntu

Zdefiniuj tablicę dwuwymiarową, która będzie reprezentować labirynt:

- Wszystkie komórki powinny być zainicjalizowane na `-1` (ściany).
 - Komórki odpowiadające korytarzom powinny być ustawione na `0`.
 - Cykle, aby wątki mogły rywalizować.
-

Krok 2: Zainicjalizuj tablicę blokad

Przygotuj tablicę blokad (`lck`), która będzie zabezpieczać operacje na poszczególnych komórkach:

- Dla każdej komórki korytarza (`0`) przypisz blokadę.
 - Użyj funkcji `omp_init_lock()` do zainicjalizowania blokad.
-

Krok 3: Implementuj funkcję eksploracji (`lab`)

1. **Przypisanie ID zadania:** Każde nowe zadanie eksplorujące labirynt musi uzyskać unikalny identyfikator (`id`).
Przydzielanie ID musi odbywać się w sekcji krytycznej, aby zapewnić, że różne zadania nie uzyskają tego samego identyfikatora.
 2. **Eksploracja bieżącej komórki:**
 - Zadanie sprawdza, czy bieżąca komórka jest korytarzem (`0`).
 - Jeżeli tak, oznacza komórkę swoim ID. Operacje odczytu i zapisu do tablicy muszą być chronione przez odpowiednią blokadę (`lock`).
 3. **Przechodzenie do sąsiadów:**
 - Zadanie sprawdza cztery kierunki (prawo, dół, lewo, góra):
 - Jeżeli sąsiadująca komórka jest korytarzem (`0`), tworzone jest nowe zadanie, które eksploruje tę komórkę.
 - Operacje te muszą być zabezpieczone blokadami.
 4. **Synchronizacja zadań potomnych:** Zadanie czeka na zakończenie wszystkich zadań potomnych, zanim zakończy swoją pracę. Użyj `#pragma omp taskwait` do synchronizacji.
-

Krok 4: Uruchom eksplorację równoległą

1. W bloku `#pragma omp parallel` ustaw liczbę wątków i uruchom eksplorację od wybranego punktu startowego (np. `(1, 1)`).
 2. Użyj dyrektywy `#pragma omp single`, aby jedno zadanie rozpoczęło eksplorację labiryntu.
-

Krok 5: Wyświetl wynik

Po zakończeniu eksploracji wyświetl stan tablicy. Każda komórka korytarza (`0`) zostanie zastąpiona identyfikatorem zadania, które ją odwiedziło.

4. Kluczowe wskazówki

1. **Bezpieczeństwo dostępu do tablicy:**
 - Każdy odczyt lub zapis do tablicy labiryntu (`tab`) musi być chroniony przez odpowiednią blokadę.
 - Użyj `omp_set_lock()` i `omp_unset_lock()` dla operacji na komórkach.
2. **Unikalne ID zadań:**
 - Identyfikatory (`id`) muszą być przypisywane w **sekcji krytycznej**, aby różne zadania nie uzyskały tego samego identyfikatora.
3. **Użycie zadań OpenMP:**

- Tworzenie nowych zadań dla sąsiednich komórek realizuj za pomocą dyrektywy `#pragma omp task`.

4. Synchronizacja zadań:

- Użyj `#pragma omp taskwait`, aby zapewnić, że wszystkie zadania potomne zakończą się przed zakończeniem zadania rodzica.
-

5. Kompilacja i uruchomienie

1. Kompilacja programu:

Użyj kompilatora z obsługą OpenMP z flagą `-fopenmp` oraz `-lgomp`.

2. Zapis konsolowy (db), plik PPM (bdb).